

### Section 1: Setup & Configuration

#### 1. `git config --global user.name "Your Name"`

**Description:** Sets the name that will be identifiable for credit when reviewing version history.

#### 2. `git config --global user.email "your.email@example.com"`

**Description:** Sets an email address that will be associated with each history marker.

#### 3. `git config --global color.ui auto`

**Description:** Sets automatic command line coloring for Git for easy reviewing.

```
PS D:\MCA - 2\MAP\Practical1> git config --global user.name "Aryan-chaudhari-107"
PS D:\MCA - 2\MAP\Practical1> git config --global user.email "bunny.reg.6531@gmail.com"
PS D:\MCA - 2\MAP\Practical1> git config --global color.ui auto
PS D:\MCA - 2\MAP\Practical1> git config --global --list
user.email=bunny.reg.6531@gmail.com
user.name=Aryan-chaudhari-107
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
color.ui=auto
PS D:\MCA - 2\MAP\Practical1>
```

#### 4. `git init`

**Description:** Initializes an existing directory as a Git repository.

```
PS D:\MCA - 2\MAP\Practical1> mkdir Project1

Directory: D:\MCA - 2\MAP\Practical1


Mode                LastWriteTime         Length Name
----                -
d-----          20-01-2026         18:52         Project1

PS D:\MCA - 2\MAP\Practical1> cd Project1
PS D:\MCA - 2\MAP\Practical1\Project1> git init
Initialized empty Git repository in D:/MCA - 2/MAP/Practical1/Project1/.git/
PS D:\MCA - 2\MAP\Practical1\Project1>
```

## Lab – 1 (MAP) Git & GitHub

What it's contain inside **.git** folder??

```
PS D:\MCA - 2\MAP\Practical1\Project1> ls .git

Directory: D:\MCA - 2\MAP\Practical1\Project1\.git

Mode                LastWriteTime         Length Name
----                -
d-----         20-01-2026        18:52         hooks
d-----         20-01-2026        18:52         info
d-----         20-01-2026        18:52        objects
d-----         20-01-2026        18:52         refs
-a-----         20-01-2026        18:52        130 config
-a-----         20-01-2026        18:52         73 description
-a-----         20-01-2026        18:52         21 HEAD
```

### Section 2: Stage & Snapshot

#### 5. git status

**Description:** Shows modified files in working directory that are staged for your next commit.

```
PS D:\MCA - 2\MAP\Practical1\Project1> touch names.txt
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    names.txt

nothing added to commit but untracked files present (use "git add" to track)
PS D:\MCA - 2\MAP\Practical1\Project1>
```

It's showing **red (name.txt)** cuz it has not added (staged).

#### 6. git add [file] or git add .

**Description:** Adds a file as it looks now to your next commit (stage).

Here This dot will add all red files that are not staged. Doesn't matter how many file are there

```
PS D:\MCA - 2\MAP\Practical1\Project1> git add names.txt
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   names.txt

PS D:\MCA - 2\MAP\Practical1\Project1>
```

“Just a reference for future concept”

```
PS D:\MCA - 2\MAP\Practical1\Project1> git commit -m "names.txt file added"
[main (root-commit) a07b50e] names.txt file added
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 names.txt
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main
nothing to commit, working tree clean
PS D:\MCA - 2\MAP\Practical1\Project1>
```

## Lab – 1 (MAP) Git & GitHub

⇒ **git reset [file]**

**Description:** Unstages a file while retaining the changes in working directory.

```
PS D:\MCA - 2\MAP\Practical1\Project1> cat names.txt
Name: Aryan Chaudhari
Roll No. : 25MCA009
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   names.txt

no changes added to commit (use "git add" and/or "git commit -a")
PS D:\MCA - 2\MAP\Practical1\Project1> git add .
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   names.txt

PS D:\MCA - 2\MAP\Practical1\Project1> git reset names.txt
Unstaged changes after reset:
M       names.txt
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   names.txt

no changes added to commit (use "git add" and/or "git commit -a")
PS D:\MCA - 2\MAP\Practical1\Project1>
```

7. **git diff** (Here git diff is not showing anything cuz I have staged names.txt)

**Description:** Shows the difference of what is changed but not staged.

8. **git diff --staged**

**Description:** Shows the difference between what is staged but not yet committed.

```
PS D:\MCA - 2\MAP\Practical1\Project1> git add names.txt
PS D:\MCA - 2\MAP\Practical1\Project1> git diff
PS D:\MCA - 2\MAP\Practical1\Project1> git diff --staged
diff --git a/names.txt b/names.txt
index e69de29..f1a6f9a 100644
--- a/names.txt
+++ b/names.txt
@@ -0,0 +1,2 @@
+Name: Aryan Chaudhari
+Roll No. : 25MCA009
\ No newline at end of file
PS D:\MCA - 2\MAP\Practical1\Project1>
```

## Lab – 1 (MAP) Git & GitHub

### 9. `git commit -m "[descriptive message]"`

**Description:** Commits your staged content as a new commit snapshot. (snapshot is taken)

```
\ No newline at end of file
PS D:\MCA - 2\MAP\Practical1\Project1> git commit -m "modified names.txt"
[main ad8b2d7] modified names.txt
 1 file changed, 2 insertions(+)
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main
nothing to commit, working tree clean
PS D:\MCA - 2\MAP\Practical1\Project1> |
```

## Section 3: Commit History, Remove File & Reset Commit

### 10. git log

**Description:** Shows all commits in the current branch's history.

```
PS D:\MCA - 2\MAP\Practical1\Project1> git log
commit ad8b2d7b97bdbc5326bff5846845e68667cd7a34 (HEAD -> main)
Author: Aryan-chaudhari-107 <bunny.reg.6531@gmail.com>
Date: Tue Jan 20 19:37:03 2026 +0530

    modified names.txt

commit a07b50e2d266b95b73ef38b6f3032bcf2f59f00e
Author: Aryan-chaudhari-107 <bunny.reg.6531@gmail.com>
Date: Tue Jan 20 19:19:35 2026 +0530

    names.txt file added
PS D:\MCA - 2\MAP\Practical1\Project1>
```

### 11. git rm [file]

**Description:** Deletes the file from project and stages the removal for commit.

```
PS D:\MCA - 2\MAP\Practical1\Project1> git rm names.txt
rm 'names.txt'
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    names.txt
```

```
PS D:\MCA - 2\MAP\Practical1\Project1> git commit -m "deleted names.txt"
[main db498a7] deleted names.txt
1 file changed, 2 deletions(-)
delete mode 100644 names.txt
PS D:\MCA - 2\MAP\Practical1\Project1> git log
commit db498a7d3f04bb490a3128d92d509dab9d4e0a02 (HEAD -> main)
Author: Aryan-chaudhari-107 <bunny.reg.6531@gmail.com>
Date: Tue Jan 20 19:46:33 2026 +0530

    deleted names.txt

commit ad8b2d7b97bdbc5326bff5846845e68667cd7a34
Author: Aryan-chaudhari-107 <bunny.reg.6531@gmail.com>
Date: Tue Jan 20 19:37:03 2026 +0530

    modified names.txt

commit a07b50e2d266b95b73ef38b6f3032bcf2f59f00e -> ID
Author: Aryan-chaudhari-107 <bunny.reg.6531@gmail.com>
Date: Tue Jan 20 19:19:35 2026 +0530

    names.txt file added
PS D:\MCA - 2\MAP\Practical1\Project1>
```

## Lab – 1 (MAP) Git & GitHub

**NOTE:** I deleted this square bracket part by mistake. I didn't want to do it. I want to remove

This comment from my history

Each commit builds on top of the others

Let's say I want to go back to any particular history, like on **"Tue Jan 20 19:19:35 2026"**

**Now understand:**

If I want to rollback to here **"Tue Jan 20 19:19:35 2026"**, I will select this commit ID

(Or)

If I want to remove the upper two commit I will select the below commit (first commit)

(Just copy the commit ID)

⇒ **git reset [commit ID]**

**Description:** Moves the branch to the given commit, keeps changes in the working directory, but unstages them.

```
PS D:\MCA - 2\MAP\Practical1\Project1> git reset a07b50e2d266b95b73ef38b6f3032bcf2f59f00e
Unstaged changes after reset:
D   names.txt
PS D:\MCA - 2\MAP\Practical1\Project1> git log
commit a07b50e2d266b95b73ef38b6f3032bcf2f59f00e (HEAD -> main)
Author: Aryan-chaudhari-107 <bunny.reg.6531@gmail.com>
Date:   Tue Jan 20 19:19:35 2026 +0530

    names.txt file added
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    names.txt

no changes added to commit (use "git add" and/or "git commit -a")
PS D:\MCA - 2\MAP\Practical1\Project1>
```

⇒ **git reset --hard [commit ID]**

**Description:** Moves the branch to the given commit and discards all changes in both the staging area and working directory..

### Section 4: Temporary Commits

Now I want to save all these changes, but I do not want to commit so my history looks clean

And I also don't want to lose my changes

```
PS D:\MCA - 2\MAP\Practical1\Project1> git add .
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    deleted:    names.txt

PS D:\MCA - 2\MAP\Practical1\Project1> touch x.txt
PS D:\MCA - 2\MAP\Practical1\Project1> git add .
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    renamed:    names.txt -> x.txt

PS D:\MCA - 2\MAP\Practical1\Project1> vi x.txt
PS D:\MCA - 2\MAP\Practical1\Project1> touch houses.txt
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    renamed:    names.txt -> x.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   x.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    houses.txt

PS D:\MCA - 2\MAP\Practical1\Project1>
```

These changes are added (it's not in Screen Shot). Now I'm going to perfume temporary commits



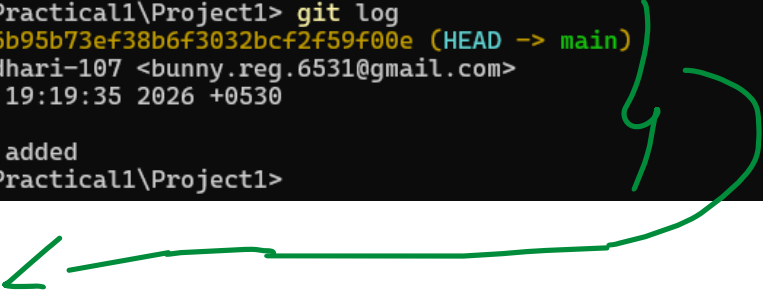
## Lab – 1 (MAP) Git & GitHub

### 12. git stash

**Description:** Saves modified and staged changes.

```
PS D:\MCA - 2\MAP\Practical1\Project1> git stash
Saved working directory and index state WIP on main: a07b50e names.txt file added
PS D:\MCA - 2\MAP\Practical1\Project1> git status
On branch main
nothing to commit, working tree clean
PS D:\MCA - 2\MAP\Practical1\Project1> git log
commit a07b50e2d266b95b73ef38b6f3032bcf2f59f00e (HEAD -> main)
Author: Aryan-chaudhari-107 <bunny.reg.6531@gmail.com>
Date: Tue Jan 20 19:19:35 2026 +0530

    names.txt file added
PS D:\MCA - 2\MAP\Practical1\Project1>
```



(Same as Previous)

### 13. git stash pop

**Description:** Writes working from top of stash stack.

```
PS D:\MCA - 2\MAP\Practical1\Project1> git stash pop
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   houses.txt

Dropped refs/stash@{0} (219889cffee6d22292f5ca5bb94e32bc7e2216ab)
PS D:\MCA - 2\MAP\Practical1\Project1>
```

### 14. git stash list

**Description:** Lists the stack order of stashed file changes.

### 15. git stash clear

**Description:** clear everything from the stash

```
PS D:\MCA - 2\MAP\Practical1\Project1> git stash list
stash@{0}: WIP on main: a07b50e names.txt file added
stash@{1}: WIP on main: a07b50e names.txt file added
PS D:\MCA - 2\MAP\Practical1\Project1> git stash clear
PS D:\MCA - 2\MAP\Practical1\Project1> git stash list
PS D:\MCA - 2\MAP\Practical1\Project1>
```

### Section 5: Share & Update

#### 16. git remote add [alias] [url]

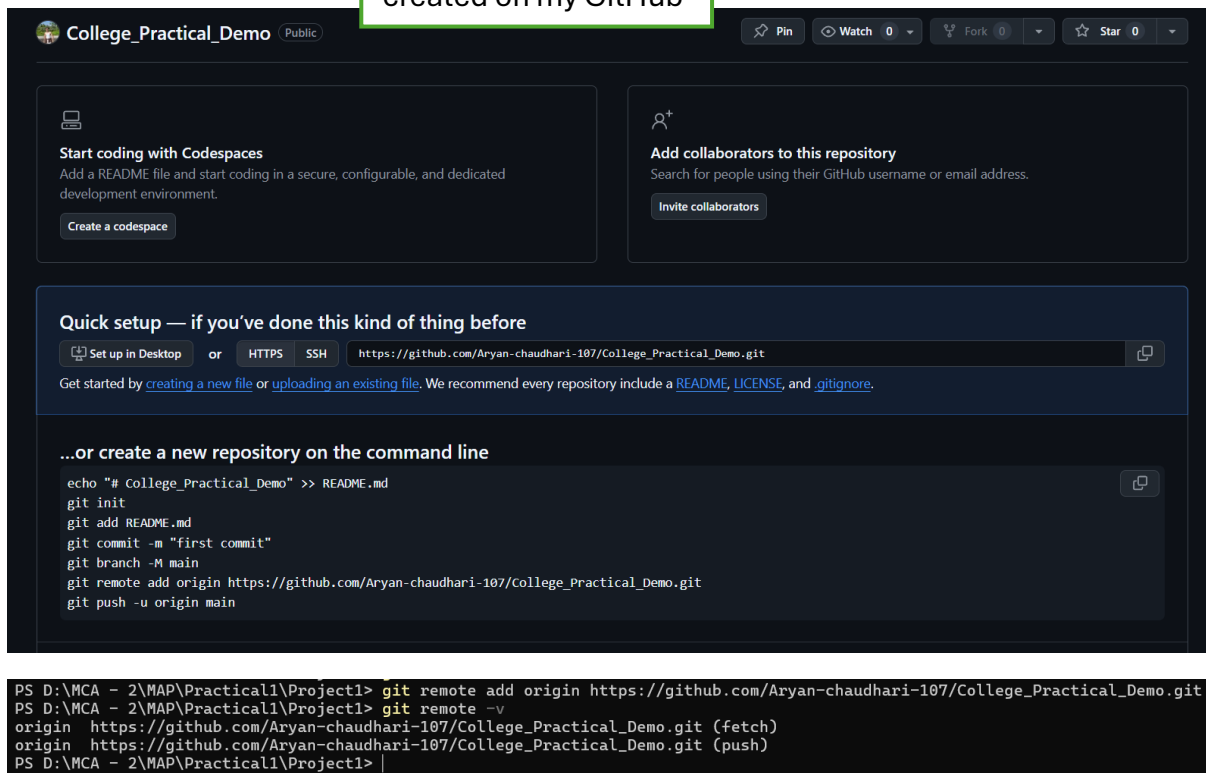
**Description:** Adds a git URL as an alias

If no alias is chosen, origin is conventionally used as the default name for the remote URL.

#### 17. git remote -v

**Description:** displays the list of configured remote repositories along with their fetch and push URLs.

Repo, which I have  
created on my GitHub



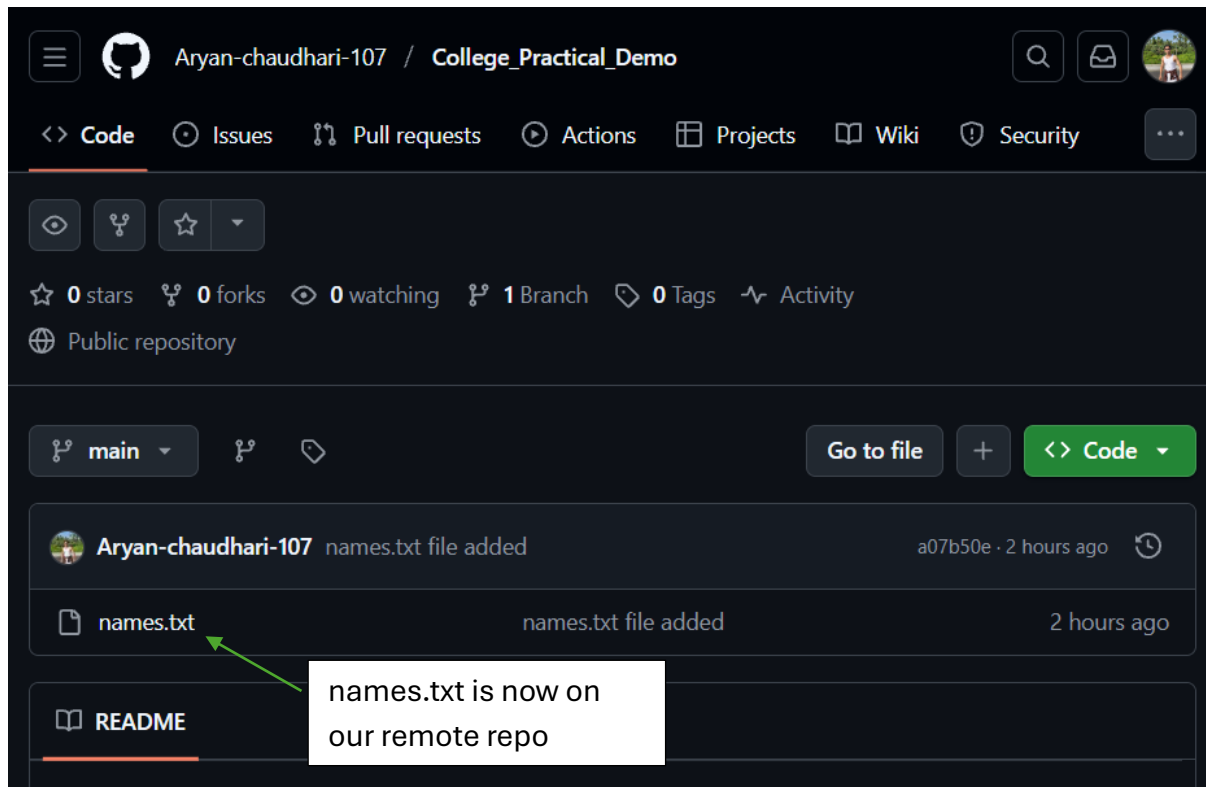
The screenshot shows the GitHub interface for a repository named 'College\_Practical\_Demo'. At the top, there are buttons for 'Pin', 'Watch' (0), 'Fork' (0), and 'Star' (0). Below this, there are two main sections: 'Start coding with Codespaces' and 'Add collaborators to this repository'. The 'Quick setup' section provides instructions for cloning the repository using HTTPS or SSH, with the URL 'https://github.com/Aryan-chaudhari-107/College\_Practical\_Demo.git'. Below this, there is a section for creating a new repository on the command line, which includes a list of commands: 'git init', 'git add README.md', 'git commit -m "first commit"', 'git branch -M main', 'git remote add origin https://github.com/Aryan-chaudhari-107/College\_Practical\_Demo.git', and 'git push -u origin main'. At the bottom, there is a terminal window showing the output of the 'git remote add origin' command, which is 'origin https://github.com/Aryan-chaudhari-107/College\_Practical\_Demo.git (push)'.

#### 18. git push [alias] [branch]

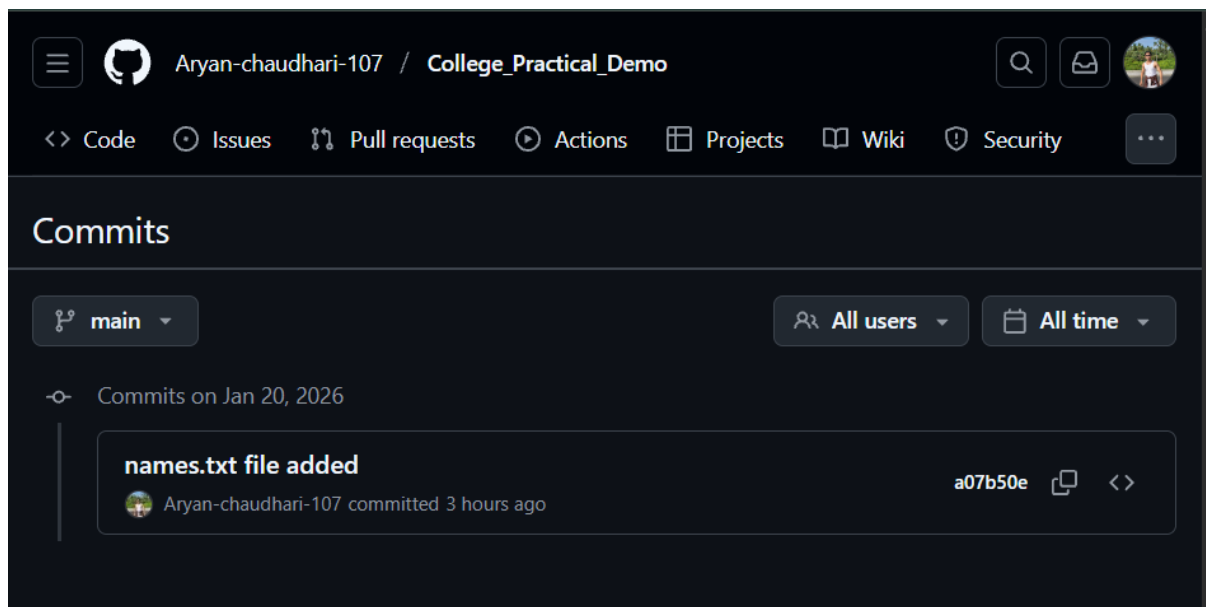
**Description:** Transmits local branch commits to the remote repository branch.

```
PS D:\MCA - 2\MAP\Practical1\Project1> git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 228 bytes | 228.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Aryan-chaudhari-107/College_Practical_Demo.git
 * [new branch]      main -> main
PS D:\MCA - 2\MAP\Practical1\Project1> |
```

## Lab – 1 (MAP) Git & GitHub



This is the only one commit that I have pushed on remote repo.



I have created some random txt files and committed on local repo, but I have not pushed, so they are limited to the local repo

Which you can understand from the screenshot below: look, there is the original main on commit that I have pushed on the remote repo.

## Lab – 1 (MAP) Git & GitHub

```
log
PS D:\MCA - 2\MAP\Practical1\Project1> git log
commit ffadd967f2bf4f0d7fd65d7eb162de2185d1d4a (HEAD -> main)
Author: Aryan-chaudhari-107 <bunny.reg.6531@gmail.com>
Date: Tue Jan 20 22:17:11 2026 +0530

    hotel.txt and rollno.txt modified

commit 311f740abf8a5e907e7b548c8eb43f8c6b2a0120
Author: Aryan-chaudhari-107 <bunny.reg.6531@gmail.com>
Date: Tue Jan 20 22:15:44 2026 +0530

    rollno.txt added

commit dd78fce978d7c8bf621590306868e4594e4f9648
Author: Aryan-chaudhari-107 <bunny.reg.6531@gmail.com>
Date: Tue Jan 20 22:15:20 2026 +0530

    hotel.txt added

commit a07b50e2d266b95b73ef38b6f3032bcf2f59f00e (origin/main)
Author: Aryan-chaudhari-107 <bunny.reg.6531@gmail.com>
Date: Tue Jan 20 19:19:35 2026 +0530

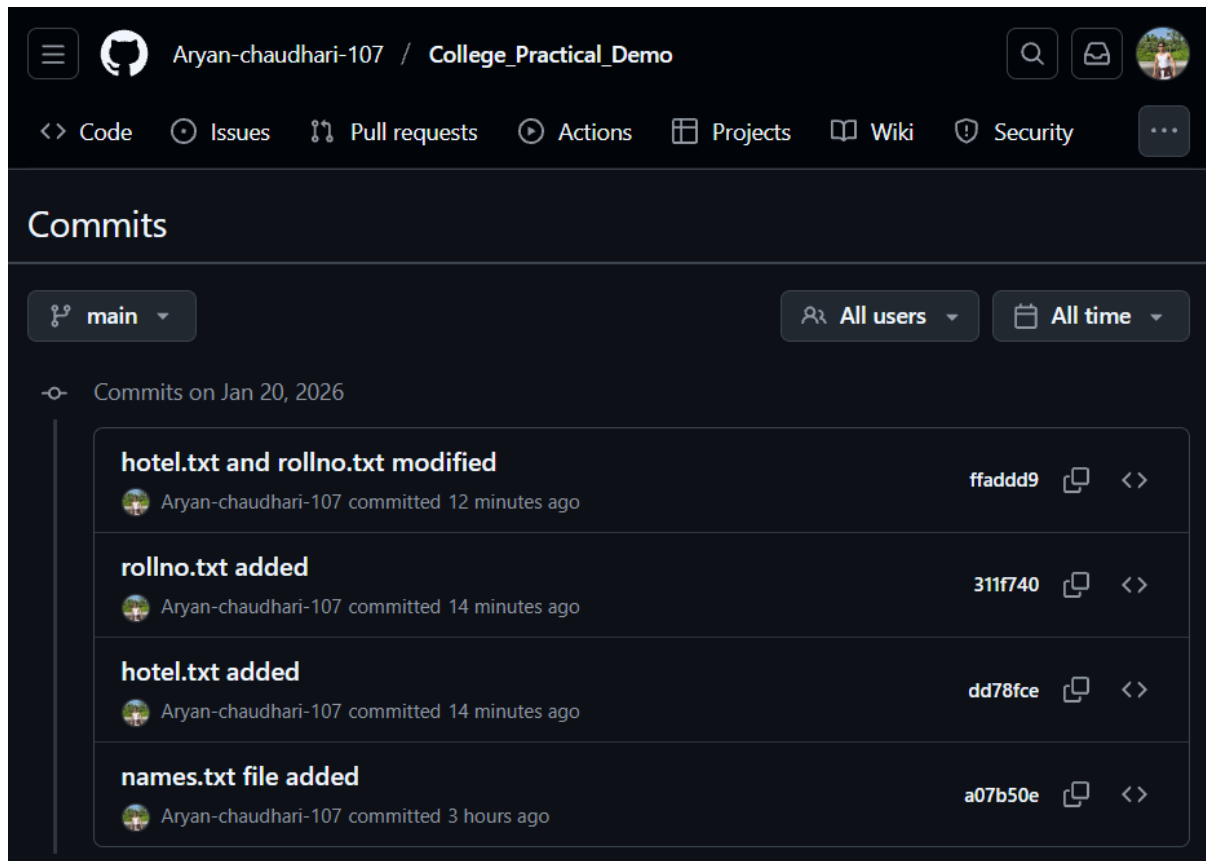
    names.txt file added
PS D:\MCA - 2\MAP\Practical1\Project1> |
```

Now I want to push this commit on main branch on remote repo

```
names.txt file added
PS D:\MCA - 2\MAP\Practical1\Project1> git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (8/8), 773 bytes | 773.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/Aryan-chaudhari-107/College_Practical_Demo.git
    a07b50e..ffadd9  main -> main
PS D:\MCA - 2\MAP\Practical1\Project1> |
```

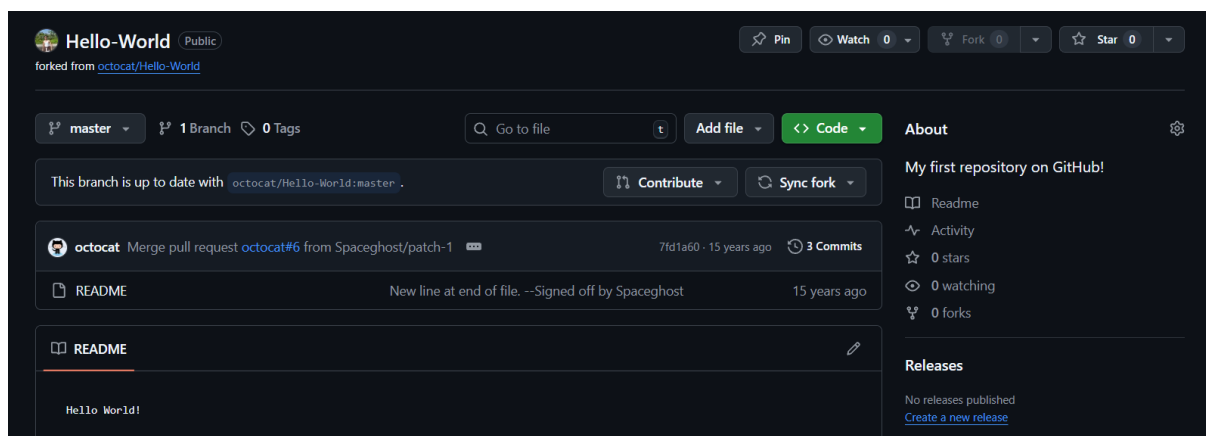
In the screenshot below, you can see all commit pushed on remote repo:

## Lab – 1 (MAP) Git & GitHub



### 19. Fork

**Description:** A fork is a personal copy of someone else's repository created under your own GitHub account, allowing you to freely experiment, modify code, and propose changes without affecting the original project.



I have forked octocat/Hello-World successfully.

## Lab – 1 (MAP) Git & GitHub

### 20. git clone [url]

**Description:** Retrieves an entire repository from a hosted location via URL.

```
PS D:\MCA - 2\MAP\Practical1\Project1> git clone https://github.com/Aryan-chaudhari-107/Hello-World.git
Cloning into 'Hello-World'...
remote: Enumerating objects: 7, done.
remote: Total 7 (delta 0), reused 0 (delta 0), pack-reused 7 (from 1)
Receiving objects: 100% (7/7), done.
PS D:\MCA - 2\MAP\Practical1\Project1> |
```

README.md Before:

```
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> cat README
Hello World!
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> |
```

README.md After:

```
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> cat README
Hello World!
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README

no changes added to commit (use "git add" and/or "git commit -a")
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> cat README
Hello World!

By: 25MCA009
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> |
```

## Section 6: Branch & Merge

**“Do not commit directly on the main branch”**

### 21. git branch

**Description:** Used to list, create, or delete branches.

Does not switch to another branch.

### 22. git checkout

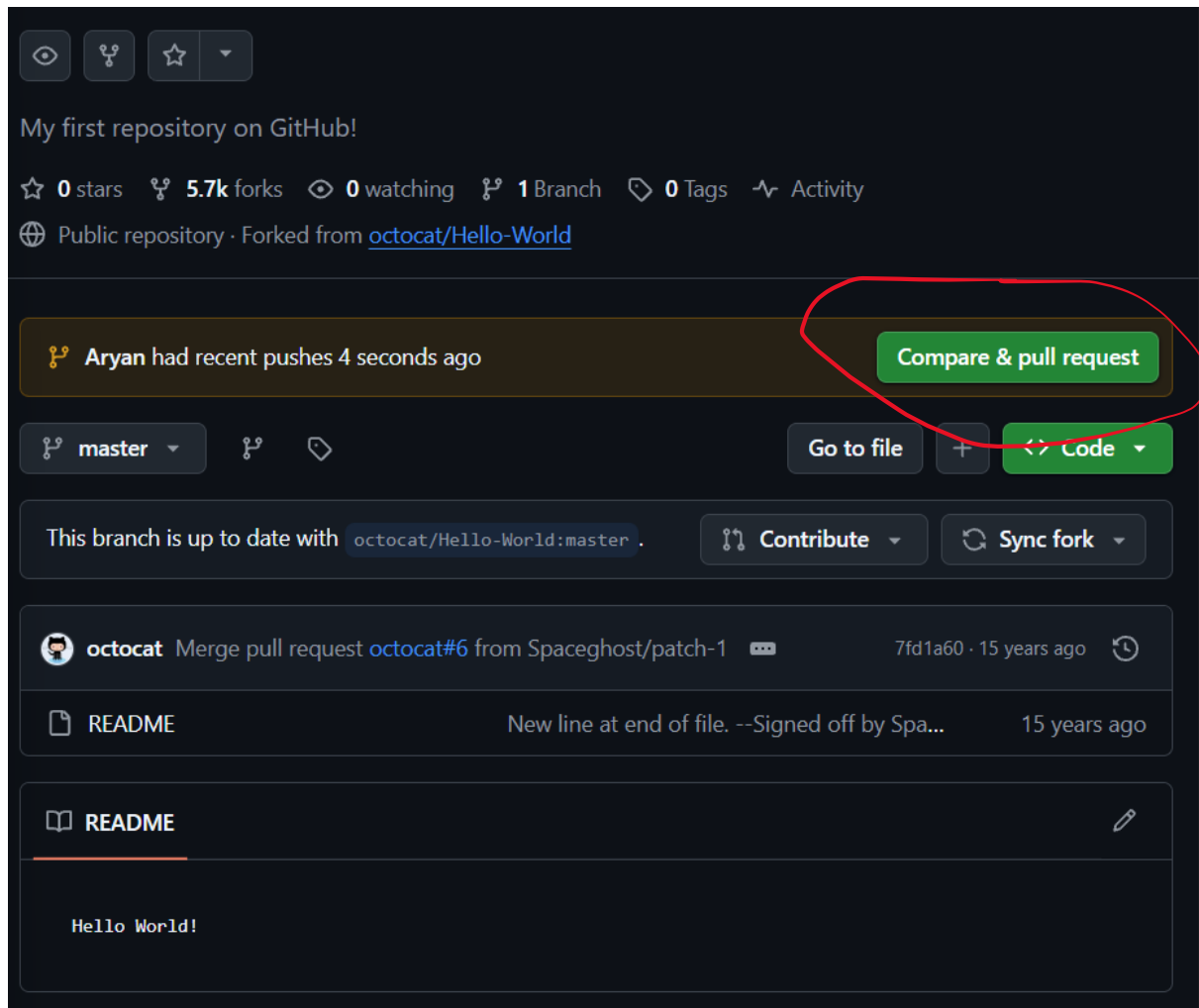
**Description:** Used to switch between branches or restore files.

```
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> git branch Aryan
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> git checkout Aryan
M      README
Switched to branch 'Aryan'
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> |
```

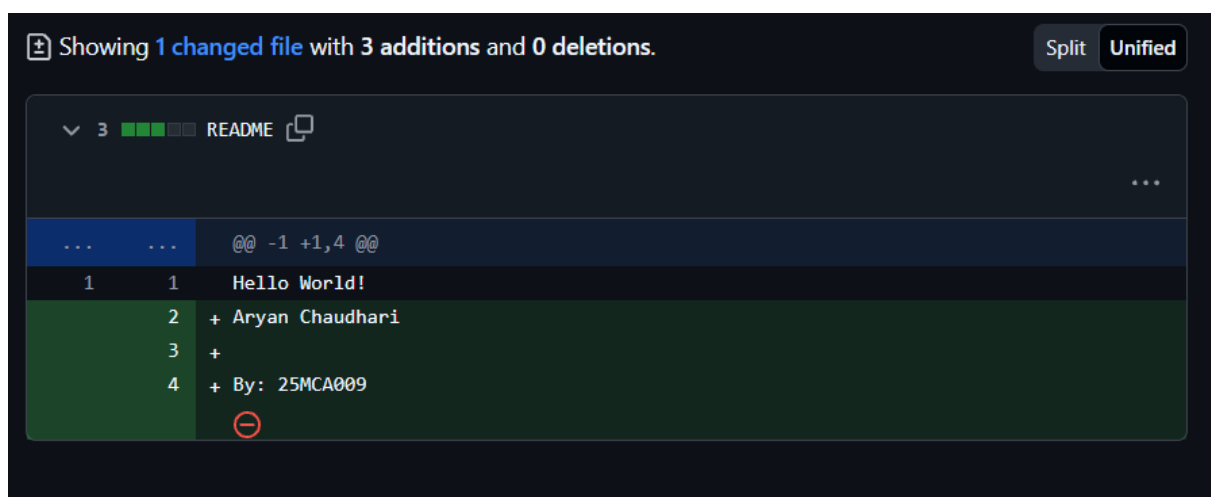
**Add ., committed and pushed on Aryan branch**

```
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> git add .
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> git commit -m Aryan
[Aryan 624b1da] Aryan
 1 file changed, 2 insertions(+)
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> git commit -m "Aryan added a message"
On branch Aryan
nothing to commit, working tree clean
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World> git push origin Aryan
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 270 bytes | 270.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'Aryan' on GitHub by visiting:
remote:   https://github.com/Aryan-chaudhari-107/Hello-World/pull/new/Aryan
remote:
To https://github.com/Aryan-chaudhari-107/Hello-World.git
 * [new branch]      Aryan -> Aryan
PS D:\MCA - 2\MAP\Practical1\Project1\Hello-World>
```

## Lab – 1 (MAP) Git & GitHub



These are the changes (shown below):



One pull request means one branch, one branch only open one request.

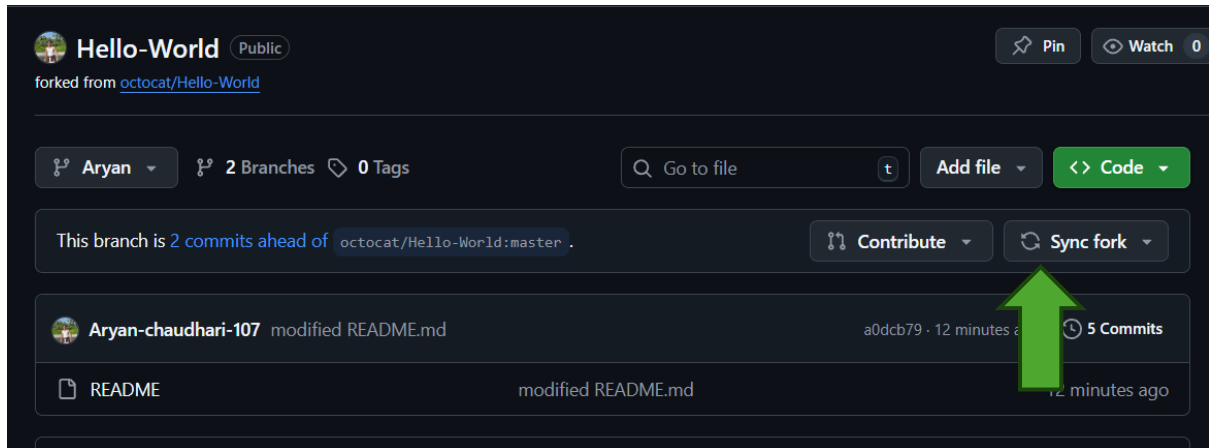


## Lab – 1 (MAP) Git & GitHub

### 23. Sync fork:

**Description:** The Sync fork button is used to update your forked repository with the latest changes from the original (upstream) repository.

**Sync fork** = fetch + merge (on GitHub UI)



Also we can use commands like:

⇒ **git fetch upstream**

**Description:** Downloads updates from a remote repository **without merging** them into your current branch.

⇒ **git merge upstream/main**

**Description:** Combines another branch into the current branch.

⇒ **git pull upstream main**

**Description:** Fetches updates from a remote repository **and merges** them into your current branch.

